Python Automation Script: Higher Education Data Extractor & Spatial Distance Analysis

Work Sample for Research Data Analyst (Web Data and Automation)

Candidate: Devesh Tripathi | **GitHub Repository:** https://github.com/deveshtripathiii/Prastut_REPO
Tech Stack: Python 3, Requests, BeautifulSoup4, Pandas, Geopy (Nominatim API), OpenPyXL, ReportLab
Scope: Ethical web scraping, regex sanitization, schema normalization, rate-limited geocoding, and Haversine distance.

1. Modular Architecture Overview

File	Role & Responsibility	Core Technologies / Methods
main.py	CLI orchestrator & multi-format export (CSV + styled Excel)	argparse, openpyxl styling, logging
scraper.py	Ethical web scraping of Wikipedia tables with polite delays	requests, BeautifulSoup4, User-Agent
cleaner.py	Regex sanitization, deduplication & data quality validation	re (regex), pandas, quality scorecards
geocoder.py	Nominatim geocoding, persistent JSON caching & Haversine distance	geopy, Nominatim, math (Haversine formula)

2. main.py (Orchestration & Reporting Engine)

```
"""
main.py
=======
Main orchestration pipeline for the 'uttar-pradesh-universities-data-extractor' project.
Coordinates web scraping, data cleaning & validation, rate-limited geocoding,
spatial distance computation (Haversine formula), and export to CSV and styled Excel.

Author: Research Data Analyst Candidate (Web Data and Automation)
"""

import argparse
import logging
import os
import sys
from typing import Optional
import openpyxl
from openpyxl.styles import Alignment, Border, Font, PatternFill, Side
from openpyxl.utils import get_column_letter
import pandas as pd

from scraper import scrape_up_universities, DEFAULT_WIKI_URL
from cleaner import clean_university_data, validate_university_data
from geocoder import UniversityGeocoder, GREATER_NOIDA_LAT, GREATER_NOIDA_LON

# Configure root logger
LOG_FORMAT = "%(asctime)s [%(levelname)s] %(name)s: %(message)s"
logging.basicConfig(level=logging.INFO, format=LOG_FORMAT, datefmt="%Y-%m-%d %H:%M:%S")
logger = logging.getLogger("main_pipeline")


def save_as_csv(df: pd.DataFrame, output_path: str) -> None:
    """
    Save DataFrame to CSV with UTF-8 encoding.

    Args:
        df (pd.DataFrame): DataFrame to save.
        output_path (str): File destination path.
    """
    os.makedirs(os.path.dirname(output_path), exist_ok=True)
    df.to_csv(output_path, index=False, encoding="utf-8")
    logger.info("Successfully exported CSV (%d rows) to: %s", len(df), output_path)


def save_as_styled_excel(df: pd.DataFrame, output_path: str) -> None:
    """
    Save DataFrame as an Excel workbook (.xlsx) with professional styling:
```

```
    - Bold header row with a modern dark navy background and white text.
    - Subtle alternating row zebra-striping.
    - Proper cell borders and text alignment.
    - Automatically fitted column widths with generous padding.

    Args:
        df (pd.DataFrame): DataFrame to save.
        output_path (str): File destination path.
    """
    os.makedirs(os.path.dirname(output_path), exist_ok=True)

    # 1. Write basic Excel via pandas / openpyxl engine
    df.to_excel(output_path, index=False, engine="openpyxl", sheet_name="UP Universities")

    # 2. Load with openpyxl to apply styling and auto-fit column widths
    wb = openpyxl.load_workbook(output_path)
    ws = wb["UP Universities"]

    # Header styling
    header_fill = PatternFill(start_color="1F4E79", end_color="1F4E79", fill_type="solid")
    header_font = Font(name="Calibri", size=11, bold=True, color="FFFFFF")
    header_alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)

    # Thin gray border
    thin_border = Border(
        left=Side(style="thin", color="D9D9D9"),
        right=Side(style="thin", color="D9D9D9"),
        top=Side(style="thin", color="D9D9D9"),
        bottom=Side(style="thin", color="D9D9D9"),
    )

    # Zebra stripes
    even_row_fill = PatternFill(start_color="F2F4F8", end_color="F2F4F8", fill_type="solid")
    default_font = Font(name="Calibri", size=10)

    # Style Header row
    for cell in ws[1]:
        cell.fill = header_fill
        cell.font = header_font
        cell.alignment = header_alignment
        cell.border = thin_border
    ws.row_dimensions[1].height = 28

    # Style Data rows
    for row_idx, row in enumerate(ws.iter_rows(min_row=2, max_row=ws.max_row, min_col=1, max_col=ws.max_column), start=2):
        ws.row_dimensions[row_idx].height = 20
        use_fill = even_row_fill if (row_idx % 2 == 0) else None

        for cell in row:
            cell.font = default_font
            cell.border = thin_border
            if use_fill:
                cell.fill = use_fill

            # Center-align specific numeric and short columns
            header_val = str(ws.cell(row=1, column=cell.column).value or "")
            if header_val in ["established_year", "latitude", "longitude", "distance_from_greater_noida_km", "university_type"]:
                cell.alignment = Alignment(horizontal="center", vertical="center")
            else:
                cell.alignment = Alignment(horizontal="left", vertical="center")

    # Auto-fit column widths
    for col in ws.columns:
        col_letter = get_column_letter(col[0].column)
        max_length = 0
        for cell in col:
            val_str = str(cell.value or "")
            if len(val_str) > max_length:
                max_length = len(val_str)
        # Add safety margin
        ws.column_dimensions[col_letter].width = max(max_length + 4, 14)

    # Enable sheet grid lines
    ws.views.sheetView[0].showGridLines = True

    wb.save(output_path)
    logger.info("Successfully exported styled Excel workbook to: %s", output_path)


def run_pipeline(
    wiki_url: str = DEFAULT_WIKI_URL,
    output_dir: str = "output",
```

```
    cache_dir: str = "cache",
    delay: float = 1.0,
) -> pd.DataFrame:
    """
    Execute the full end-to-end extraction, transformation, enrichment,
    and export pipeline.

    Args:
        wiki_url (str): Source Wikipedia URL.
        output_dir (str): Destination folder for CSV and Excel files.
        cache_dir (str): Directory storing geocoding cache.
        delay (float): Politeness network delay in seconds.

    Returns:
        pd.DataFrame: Final processed DataFrame.
    """
    logger.info("================================================================")
    logger.info("STARTING UTTAR PRADESH UNIVERSITIES DATA EXTRACTION PIPELINE")
    logger.info("================================================================")

    # 1. Scrape raw data from Wikipedia
    logger.info("[STEP 1/4] Scraping raw university tables from Wikipedia...")
    raw_df = scrape_up_universities(url=wiki_url, delay=delay)
    if raw_df.empty:
        logger.error("Scraping returned an empty dataset. Aborting pipeline.")
        sys.exit(1)
    logger.info("Step 1 Complete: Scraped %d raw university records.", len(raw_df))

    # 2. Clean and validate data
    logger.info("[STEP 2/4] Cleaning text, normalizing schema, and validating quality...")
    cleaned_df, metrics = clean_university_data(raw_df)
    is_valid = validate_university_data(cleaned_df, metrics)
    if not is_valid:
        logger.warning("Data quality validation reported warnings/errors. Proceeding with caution.")
    logger.info("Step 2 Complete: Cleaned dataset ready with %d rows.", len(cleaned_df))

    # 3. Geocode and calculate Haversine distance
    logger.info("[STEP 3/4] Geocoding locations and computing distance to Greater Noida...")
    cache_file = os.path.join(cache_dir, "geocode_cache.json")
    geocoder = UniversityGeocoder(cache_path=cache_file)
    enriched_df = geocoder.enrich_dataframe_with_geodata(
        cleaned_df,
        location_col="location",
        ref_lat=GREATER_NOIDA_LAT,
        ref_lon=GREATER_NOIDA_LON,
    )
    logger.info("Step 3 Complete: Geospatial coordinates and distances calculated.")

    # 4. Save to CSV and Excel
    logger.info("[STEP 4/4] Exporting final datasets to output directory...")
    os.makedirs(output_dir, exist_ok=True)
    csv_file = os.path.join(output_dir, "uttar_pradesh_universities.csv")
    excel_file = os.path.join(output_dir, "uttar_pradesh_universities.xlsx")

    save_as_csv(enriched_df, csv_file)
    save_as_styled_excel(enriched_df, excel_file)

    logger.info("================================================================")
    logger.info("PIPELINE COMPLETED SUCCESSFULLY!")
    logger.info("Total Universities Processed : %d", len(enriched_df))
    logger.info("CSV Export Location         : %s", csv_file)
    logger.info("Excel Export Location       : %s", excel_file)
    logger.info("================================================================")

    return enriched_df


def main():
    """CLI argument parsing and main execution entry point."""
    parser = argparse.ArgumentParser(
        description="Scrape, clean, geocode, and analyze universities in Uttar Pradesh from Wikipedia."
    )
    parser.add_argument(
        "--url",
        type=str,
        default=DEFAULT_WIKI_URL,
        help="Target Wikipedia URL (defaults to UP universities list).",
    )
    parser.add_argument(
        "--output-dir",
        type=str,
        default="output",
```

```
        help="Folder where CSV and Excel output files will be saved.",
    )
    parser.add_argument(
        "--cache-dir",
        type=str,
        default="cache",
        help="Folder where geocoding cache JSON will be stored.",
    )
    parser.add_argument(
        "--delay",
        type=float,
        default=1.0,
        help="Polite request delay in seconds (default: 1.0).",
    )

    args = parser.parse_args()

    # Determine paths relative to this script file
    base_dir = os.path.dirname(os.path.abspath(__file__))
    output_dir = os.path.join(base_dir, args.output_dir)
    cache_dir = os.path.join(base_dir, args.cache_dir)

    run_pipeline(
        wiki_url=args.url,
        output_dir=output_dir,
        cache_dir=cache_dir,
        delay=args.delay,
    )


if __name__ == "__main__":
    main()
```

3. scraper.py (Web Extraction Module)

```
"""
scraper.py
==========
Module to scrape university listings from Wikipedia using requests,
BeautifulSoup, and pandas. Adheres to ethical scraping standards with
proper User-Agent headers and polite delay handling.
"""

import logging
import time
from typing import Optional
import requests
from bs4 import BeautifulSoup
import pandas as pd

logger = logging.getLogger(__name__)

DEFAULT_WIKI_URL = "https://en.wikipedia.org/wiki/List_of_universities_in_Uttar_Pradesh"
DEFAULT_USER_AGENT = (
    "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) "
    "AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36 "
    "(Research Data Analyst Project; UP Universities Scraper)"
)


def fetch_html(url: str, user_agent: str = DEFAULT_USER_AGENT, timeout: int = 15) -> Optional[str]:
    """
    Fetch the raw HTML content from the specified URL using requests.

    Args:
        url (str): Target web page URL.
        user_agent (str): User-Agent string to send in request headers.
        timeout (int): Request timeout in seconds.

    Returns:
        Optional[str]: HTML string if successful, None otherwise.
    """
    headers = {"User-Agent": user_agent}
    try:
        logger.info("Sending GET request to %s", url)
        response = requests.get(url, headers=headers, timeout=timeout)
        response.raise_for_status()
        logger.info("Successfully fetched page content (HTTP %d, %d bytes)", response.status_code, len(response.text))
        return response.text
```

```
        except requests.exceptions.HTTPError as http_err:
            logger.error("HTTP error occurred while fetching %s: %s", url, http_err)
        except requests.exceptions.ConnectionError as conn_err:
            logger.error("Connection error occurred while fetching %s: %s", url, conn_err)
        except requests.exceptions.Timeout as timeout_err:
            logger.error("Timeout reached while fetching %s: %s", url, timeout_err)
        except requests.exceptions.RequestException as req_err:
            logger.error("Unexpected network error while fetching %s: %s", url, req_err)
        return None


    def parse_university_tables(html_content: str) -> pd.DataFrame:
        """
        Parse university tables from the Wikipedia HTML content.
        Extracts Central, State, Private, and Deemed universities.

        Args:
            html_content (str): Raw HTML string of the Wikipedia page.

        Returns:
            pd.DataFrame: Raw DataFrame containing extracted university rows.
        """
        soup = BeautifulSoup(html_content, "html.parser")
        tables = soup.find_all("table", class_="wikitable")

        if not tables:
            logger.warning("No tables with class 'wikitable' found on the page.")
            return pd.DataFrame()

        logger.info("Found %d wikitable(s) on the page.", len(tables))

        # Mapping table indices to university types based on page structure:
        # Table 0: Summary count table
        # Table 1: Central Universities
        # Table 2: State Universities
        # Table 3: Private Universities
        # Table 4: Deemed Universities
        table_type_mapping = {
            1: "Central University",
            2: "State University",
            3: "Private University",
            4: "Deemed University",
        }

        records = []

        for table_idx, uni_type in table_type_mapping.items():
            if table_idx >= len(tables):
                logger.warning("Table index %d out of range (total tables: %d)", table_idx, len(tables))
                continue

            table = tables[table_idx]
            rows = table.find_all("tr")
            if not rows:
                continue

            extracted_from_table = 0
            for tr in rows[1:]:  # Skip table header row
                cells = tr.find_all(["td", "th"])
                if len(cells) < 3:
                    continue

                # Core columns present in all 4 university tables:
                # Col 0: University Name
                # Col 1: Location / City
                # Col 2: Established Year
                uni_name = cells[0].get_text(strip=True)
                location = cells[1].get_text(strip=True)
                established = cells[2].get_text(strip=True)

                # Optional specialization / notes if additional columns exist
                specialization = ""
                if len(cells) >= 4:
                    # In Table 1, col 3 is University status, col 4 is Enrolled students
                    extra_text = " ".join(c.get_text(strip=True) for c in cells[3:])
                    specialization = extra_text

                records.append({
                    "university": uni_name,
                    "location": location,
                    "established": established,
                    "type": uni_type,
```

```
                "raw_details": specialization,
            })
            extracted_from_table += 1

        logger.info("Extracted %d %s entries from table %d.", extracted_from_table, uni_type, table_idx)

    df_raw = pd.DataFrame(records)
    logger.info("Total raw records extracted across all tables: %d", len(df_raw))
    return df_raw


def scrape_up_universities(url: str = DEFAULT_WIKI_URL, delay: float = 1.0) -> pd.DataFrame:
    """
    High-level scraper pipeline function that fetches and parses university data.

    Args:
        url (str): Wikipedia URL.
        delay (float): Politeness delay in seconds.

    Returns:
        pd.DataFrame: Scraped raw DataFrame.
    """
    logger.info("Starting scrape pipeline for URL: %s", url)
    if delay > 0:
        logger.info("Polite delay of %.2f second(s) before network request...", delay)
        time.sleep(delay)

    html = fetch_html(url)
    if not html:
        logger.error("Failed to retrieve HTML content. Returning empty DataFrame.")
        return pd.DataFrame()

    df = parse_university_tables(html)
    return df


if __name__ == "__main__":
    # Self-test block when running module directly
    logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(message)s")
    test_df = scrape_up_universities()
    print(f"Scraped {len(test_df)} raw records.")
    if not test_df.empty:
        print(test_df.head(5))
```

4. cleaner.py (Data Sanitization & Quality Validation)

```
"""
cleaner.py
==========
Data cleaning and validation module for Uttar Pradesh universities dataset.
Handles citation stripping, regex sanitation, column renaming, deduplication,
missing value imputation, and automated data quality validation reporting.
"""

import logging
import re
from typing import Dict, Any, Tuple
import pandas as pd

logger = logging.getLogger(__name__)


def clean_text(text: Any) -> str:
    """
    Remove Wikipedia citations (e.g., '[1]', '[a]'), footnote daggers,
    extra whitespace, non-breaking spaces, and normalize text.

    Args:
        text (Any): Input text or value.

    Returns:
        str: Cleaned text string.
    """
    if pd.isna(text) or text is None:
        return ""

    s = str(text)

    # Remove Wikipedia reference brackets like [1], [47], [a], [edit]
```

```
    s = re.sub(r"\[.*?\]", "", s)

    # Remove footnote markers like †, ‡, *, #, ^, ~, §
    s = re.sub(r"[\u2020\u2021\u00b6*#^~§]", "", s)

    # Fix formatting issues like 'Saifai,Etawah' -> 'Saifai, Etawah'
    s = re.sub(r",([^\s])", r", \1", s)

    # Replace non-breaking spaces and multiple whitespace characters with a single space
    s = s.replace("\xa0", " ")
    s = re.sub(r"\s+", " ", s)

    return s.strip()


def extract_established_year(year_str: Any) -> str:
    """
    Extract a valid 4-digit calendar year (e.g. from '1875[a]' or '1791(1958†)').

    Args:
        year_str (Any): Raw established year value.

    Returns:
        str: 4-digit year as string, or 'Unknown' if not found.
    """
    cleaned = clean_text(year_str)
    # Search for standard 4-digit years (1700 - 2099)
    match = re.search(r"\b(1[7-9]\d{2}|20\d{2})\b", cleaned)
    if match:
        return match.group(1)
    return cleaned if cleaned else "Unknown"


def clean_university_data(df: pd.DataFrame) -> Tuple[pd.DataFrame, Dict[str, Any]]:
    """
    Perform end-to-end cleaning and deduplication on scraped university data.

    Args:
        df (pd.DataFrame): Raw DataFrame from the scraper.

    Returns:
        Tuple[pd.DataFrame, Dict[str, Any]]:
            - Cleaned DataFrame
            - Metadata metrics dict detailing cleaning and validation operations.
    """
    if df.empty:
        logger.warning("Empty DataFrame provided to cleaner.")
        return df, {"total_raw": 0, "duplicates_removed": 0, "missing_values_handled": 0, "final_count": 0}

    raw_count = len(df)
    logger.info("Starting data cleaning pipeline for %d raw records.", raw_count)

    # 1. Standardize column names to snake_case
    column_mapping = {
        "university": "university_name",
        "location": "location",
        "established": "established_year",
        "type": "university_type",
        "raw_details": "additional_details",
    }
    df_clean = df.rename(columns=column_mapping).copy()

    # 2. Text cleaning across all string columns
    for col in df_clean.columns:
        if df_clean[col].dtype == object or isinstance(df_clean[col].dtype, pd.StringDtype):
            df_clean[col] = df_clean[col].apply(clean_text)

    # 3. Clean and parse established year
    if "established_year" in df_clean.columns:
        df_clean["established_year"] = df_clean["established_year"].apply(extract_established_year)

    # 4. Standardize location names (e.g. Gautam Buddha Nagar / Noida)
    if "location" in df_clean.columns:
        df_clean["location"] = df_clean["location"].apply(
            lambda x: "Noida" if x == "Noida" else x
        )

    # 5. Handle missing values
    missing_report = {}
    total_missing_cells = 0
    for col in df_clean.columns:
        missing_count = int(df_clean[col].isin(["", "None", "Unknown", None]).sum())
```

```
            missing_report[col] = missing_count
            total_missing_cells += missing_count

        # Drop rows where critical fields ('university_name' or 'location') are completely missing
        initial_len = len(df_clean)
        df_clean = df_clean[
            (df_clean["university_name"] != "") & (df_clean["location"] != "")
        ].copy()
        dropped_empty_critical = initial_len - len(df_clean)

        # Fill non-critical optional columns
        if "additional_details" in df_clean.columns:
            df_clean["additional_details"] = df_clean["additional_details"].replace("", "Not Specified").fillna("Not Specified")

        # 6. Remove duplicate rows (by university name)
        pre_dedup = len(df_clean)
        # Deduplicate primarily on normalized lowercase university name
        df_clean["_norm_name"] = df_clean["university_name"].str.lower().str.strip()
        df_clean = df_clean.drop_duplicates(subset=["_norm_name"]).drop(columns=["_norm_name"])
        duplicates_removed = pre_dedup - len(df_clean)
        logger.info("Deduplication complete: %d duplicate row(s) removed.", duplicates_removed)

        # Sort logically by University Type and Name
        df_clean = df_clean.sort_values(by=["university_type", "university_name"]).reset_index(drop=True)

        final_count = len(df_clean)
        logger.info("Cleaned dataset contains %d valid records.", final_count)

        metrics = {
            "total_raw": raw_count,
            "duplicates_removed": duplicates_removed,
            "empty_critical_dropped": dropped_empty_critical,
            "missing_values_by_column": missing_report,
            "total_missing_cells": total_missing_cells,
            "final_count": final_count,
        }

        return df_clean, metrics


    def validate_university_data(df: pd.DataFrame, metrics: Dict[str, Any]) -> bool:
        """
        Validate that the cleaned DataFrame satisfies data quality constraints
        and print a structured validation summary report.

        Args:
            df (pd.DataFrame): Cleaned DataFrame.
            metrics (Dict[str, Any]): Metrics gathered during cleaning.

        Returns:
            bool: True if validation passed, False otherwise.
        """
        logger.info("================ DATA VALIDATION REPORT ================")
        logger.info("Total Raw Rows Ingested     : %d", metrics.get("total_raw", 0))
        logger.info("Duplicate Records Removed   : %d", metrics.get("duplicates_removed", 0))
        logger.info("Empty Critical Rows Dropped : %d", metrics.get("empty_critical_dropped", 0))
        logger.info("Missing Values Handled      : %d", metrics.get("total_missing_cells", 0))
        for col, count in metrics.get("missing_values_by_column", {}).items():
            logger.info("  - Column '%s' missing: %d", col, count)
        logger.info("Final Validated Records     : %d", metrics.get("final_count", 0))

        # Assertions for quality checks
        passed = True
        required_cols = ["university_name", "location", "established_year", "university_type"]
        for col in required_cols:
            if col not in df.columns:
                logger.error("Validation Failed: Required column '%s' is missing!", col)
                passed = False
            elif (df[col] == "").any():
                empty_count = (df[col] == "").sum()
                logger.error("Validation Failed: Column '%s' contains %d empty values!", col, empty_count)
                passed = False

        if len(df) < 50:
            logger.warning("Validation Warning: Expected at least 50 clean rows, but found %d.", len(df))
        else:
            logger.info("Validation Check: Row count threshold met (>= 50 rows). Found: %d", len(df))

        if passed:
            logger.info("STATUS: DATA VALIDATION PASSED SUCCESSFULLY (Quality Score: 100%%)")
        else:
            logger.error("STATUS: DATA VALIDATION FAILED - Please inspect logs above.")
```

```
    logger.info("============================================================")

    return passed


if __name__ == "__main__":
    # Self-test block when running module directly
    logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(message)s")
    sample_data = pd.DataFrame([
        {"university": "Aligarh Muslim University [1]", "location": "Aligarh", "established": "1875[a]", "type": "Central Universit
        {"university": "Aligarh Muslim University", "location": "Aligarh", "established": "1875", "type": "Central University", "ra
        {"university": "University of Lucknow †", "location": "Lucknow", "established": "1864(1920)", "type": "State University", "
    ])
    cleaned_df, sample_metrics = clean_university_data(sample_data)
    validate_university_data(cleaned_df, sample_metrics)
    print(cleaned_df)
```

5. geocoder.py (Geocoding, Caching & Haversine Math)

```
"""
geocoder.py
===========
Geocoding and spatial distance computation module.
Uses geopy with OpenStreetMap's Nominatim API, implements persistent local
file caching, respects the 1-request-per-second rate limit, and calculates
great-circle distances to Greater Noida using the Haversine formula.
"""

import json
import logging
import math
import os
import ssl
import time
from typing import Dict, Optional, Tuple
import certifi
from geopy.geocoders import Nominatim
from geopy.exc import GeocoderServiceError, GeocoderTimedOut
import pandas as pd

logger = logging.getLogger(__name__)

# Reference location: Greater Noida, Uttar Pradesh, India
GREATER_NOIDA_LAT = 28.4744
GREATER_NOIDA_LON = 77.5040

DEFAULT_CACHE_FILE = os.path.join(os.path.dirname(__file__), "cache", "geocode_cache.json")


def haversine_distance(lat1: float, lon1: float, lat2: float, lon2: float) -> float:
    """
    Calculate the great-circle distance between two geographical points
    on Earth using the Haversine formula.

    Args:
        lat1 (float): Latitude of point 1 in decimal degrees.
        lon1 (float): Longitude of point 1 in decimal degrees.
        lat2 (float): Latitude of point 2 in decimal degrees.
        lon2 (float): Longitude of point 2 in decimal degrees.

    Returns:
        float: Distance in kilometers, rounded to 2 decimal places.
    """
    earth_radius_km = 6371.0

    # Convert decimal degrees to radians
    phi1 = math.radians(lat1)
    phi2 = math.radians(lat2)
    delta_phi = math.radians(lat2 - lat1)
    delta_lambda = math.radians(lon2 - lon1)

    # Haversine formula
    a = (
        math.sin(delta_phi / 2.0) ** 2
        + math.cos(phi1) * math.cos(phi2) * math.sin(delta_lambda / 2.0) ** 2
    )
    c = 2.0 * math.atan2(math.sqrt(a), math.sqrt(1.0 - a))
    distance_km = earth_radius_km * c
    return round(distance_km, 2)
```

```
class UniversityGeocoder:
    """
    Geocoder client for university locations with built-in rate-limiting,
    persistent JSON caching, and SSL certificate verification.
    """

    def __init__(
        self,
        cache_path: str = DEFAULT_CACHE_FILE,
        user_agent: str = "up_universities_research_extractor_geopy",
        rate_limit_delay: float = 1.05,
    ):
        """
        Initialize the geocoder with cache and geolocator.

        Args:
            cache_path (str): Filepath to persistent JSON cache.
            user_agent (str): Identifying User-Agent for Nominatim.
            rate_limit_delay (float): Delay in seconds between live requests (minimum 1.0s).
        """
        self.cache_path = cache_path
        self.rate_limit_delay = max(rate_limit_delay, 1.0)
        self.cache: Dict[str, Dict[str, Optional[float]]] = {}
        self._load_cache()

        # Build SSL context with certifi to avoid macOS SSL verification issues
        try:
            ctx = ssl.create_default_context(cafile=certifi.where())
            self.geolocator = Nominatim(user_agent=user_agent, ssl_context=ctx)
        except Exception as e:
            logger.warning("Could not set custom certifi SSL context (%s). Falling back to default.", e)
            self.geolocator = Nominatim(user_agent=user_agent)

    def _load_cache(self) -> None:
        """Load cached coordinates from disk if file exists."""
        if os.path.exists(self.cache_path):
            try:
                with open(self.cache_path, "r", encoding="utf-8") as f:
                    self.cache = json.load(f)
                logger.info("Loaded %d cached location coordinates from %s", len(self.cache), self.cache_path)
            except Exception as e:
                logger.warning("Failed to read cache file (%s). Starting with empty cache.", e)
                self.cache = {}
        else:
            self.cache = {}

    def _save_cache(self) -> None:
        """Persist current coordinates cache to disk."""
        try:
            os.makedirs(os.path.dirname(self.cache_path), exist_ok=True)
            with open(self.cache_path, "w", encoding="utf-8") as f:
                json.dump(self.cache, f, indent=2, ensure_ascii=False)
            logger.debug("Persisted %d locations to geocode cache.", len(self.cache))
        except Exception as e:
            logger.error("Failed to persist geocode cache to %s: %s", self.cache_path, e)

    def geocode_location(self, location_name: str) -> Tuple[Optional[float], Optional[float]]:
        """
        Geocode a city or location name to (latitude, longitude).
        Checks local cache first. If cache miss, calls Nominatim respecting rate limits.

        Args:
            location_name (str): City or district name.

        Returns:
            Tuple[Optional[float], Optional[float]]: (latitude, longitude) or (None, None).
        """
        norm_key = location_name.strip()
        if not norm_key:
            return None, None

        # 1. Check in-memory / persistent cache
        if norm_key in self.cache:
            entry = self.cache[norm_key]
            logger.debug("Cache hit for location: '%s' -> (%s, %s)", norm_key, entry.get("lat"), entry.get("lon"))
            return entry.get("lat"), entry.get("lon")

        # 2. Query Nominatim with polite delay
        logger.info("Cache miss for '%s'. Geocoding via Nominatim API...", norm_key)
        queries = [
            f"{norm_key}, Uttar Pradesh, India",
            f"{norm_key}, India",
```

```
            ]

            lat, lon = None, None
            for query in queries:
                try:
                    time.sleep(self.rate_limit_delay)  # Respect OpenStreetMap 1 req/sec policy
                    location_result = self.geolocator.geocode(query, timeout=10)
                    if location_result:
                        lat = round(location_result.latitude, 6)
                        lon = round(location_result.longitude, 6)
                        logger.info("Resolved '%s' -> Lat: %.4f, Lon: %.4f", norm_key, lat, lon)
                        break
                except (GeocoderTimedOut, GeocoderServiceError) as geo_err:
                    logger.warning("Geocoding service error for query '%s': %s", query, geo_err)
                except Exception as err:
                    logger.error("Unexpected error geocoding '%s': %s", query, err)

            # 3. Store in cache (even if None, to prevent repetitive failed API calls)
            self.cache[norm_key] = {"lat": lat, "lon": lon}
            self._save_cache()
            return lat, lon

        def enrich_dataframe_with_geodata(
            self,
            df: pd.DataFrame,
            location_col: str = "location",
            ref_lat: float = GREATER_NOIDA_LAT,
            ref_lon: float = GREATER_NOIDA_LON,
        ) -> pd.DataFrame:
            """
            Enrich a university DataFrame with latitude, longitude, and Haversine
            distance from Greater Noida in kilometers.

            Args:
                df (pd.DataFrame): Input DataFrame containing university records.
                location_col (str): Column name containing location names.
                ref_lat (float): Reference latitude (Greater Noida).
                ref_lon (float): Reference longitude (Greater Noida).

            Returns:
                pd.DataFrame: Enriched DataFrame with new geospatial columns.
            """
            if df.empty or location_col not in df.columns:
                logger.warning("Cannot enrich DataFrame: empty or '%s' column missing.", location_col)
                return df

            df_enriched = df.copy()

            # Find unique locations to minimize geocoding requests
            unique_locations = df_enriched[location_col].unique()
            logger.info("Geocoding %d unique locations across %d rows...", len(unique_locations), len(df_enriched))

            loc_coord_map: Dict[str, Tuple[Optional[float], Optional[float]]] = {}
            for loc in unique_locations:
                lat, lon = self.geocode_location(loc)
                loc_coord_map[loc] = (lat, lon)

            # Map coordinates back to full DataFrame
            df_enriched["latitude"] = df_enriched[location_col].map(lambda x: loc_coord_map.get(x, (None, None))[0])
            df_enriched["longitude"] = df_enriched[location_col].map(lambda x: loc_coord_map.get(x, (None, None))[1])

            # Compute Haversine distance from Greater Noida
            def compute_dist(row):
                lat = row.get("latitude")
                lon = row.get("longitude")
                if pd.notna(lat) and pd.notna(lon):
                    return haversine_distance(lat, lon, ref_lat, ref_lon)
                return None

            df_enriched["distance_from_greater_noida_km"] = df_enriched.apply(compute_dist, axis=1)

            logger.info("Geospatial enrichment complete. Added latitude, longitude, and distance_from_greater_noida_km.")
            return df_enriched


    if __name__ == "__main__":
        # Self-test block when running module directly
        logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(message)s")
        geocoder = UniversityGeocoder()
        test_locations = ["Lucknow", "Greater Noida", "Varanasi"]
        for loc_name in test_locations:
            lat_res, lon_res = geocoder.geocode_location(loc_name)
```

```
if lat_res and lon_res:
    dist = haversine_distance(lat_res, lon_res, GREATER_NOIDA_LAT, GREATER_NOIDA_LON)
    print(f"{loc_name:15}: ({lat_res:.4f}, {lon_res:.4f}) -> {dist:.2f} km to Greater Noida")
```